10b. Macros as a Means for Optimizations

Martin Alfaro
PhD in Economics

INTRODUCTION

Customized approaches often have an edge over general-purpose built-in solutions, as they can tackle the unique challenges of a given scenario. However, the complexity of specialized techniques often deters their adoption among practitioners, who may lack the necessary expertise to implement them.

Macros offer a practical solution to bridge this gap, making specialized computational approaches more accessible to users. They're particularly well-suited for this purpose, due to their ability to take entire code blocks as inputs and transform them into an optimized execution strategy. In this way, practitioners benefit from specialized algorithms, without having to write the implementation themselves.

In the upcoming sections, the role of macros in boosting performance will be central. By leveraging them, we'll be able to effectively separate the benefits provided by an algorithm from its actual implementation. This decoupling will let us shift our focus from the nitty-gritty details of how to write algorithms to the more practical question of when to apply them. The current section in particular will concentrate on the procedure for applying macros.

USES OF MACROS

Macros can be easily identified in code by the `@` symbol preceding their name. They resemble functions, in that they take input and produce output. Their primary difference lies in what they operate on and what they can return.

Specifically, *functions operate on values*: they take evaluated expressions as arguments and return a final computed value. In contrast, *macros operate on code*: they take expressions as input and return a new transformed expression, which is then compiled and executed in place of the original macro call. This unique feature enables macros to handle tasks that functions can't. Two notable applications of macro are worth mentioning.

First, macros can be used **to simplify code**. By automating repetitive tasks and eliminating boilerplate, macros can enhance code readability and maintainability. For instance, suppose a function requires multiple slices of `x` to be converted into views. Without macros, this would involve repeatedly writing `view(x, <indices>)` for each slice, resulting in verbose and error-prone code. Instead, prepending the function definition with `@views` will automatically handle all the slice conversions for us. This is demonstrated below.

NO MACRO

```
x = rand(1_000)

function foo(x)
    x1 = view(x, x .> 0.7)
    x2 = view(x, x .< 0.5)
    x3 = view(x, 1:500)
    x4 = view(x, 501:1_000)

    x1, x2, x3, x4
end
```

MACRO @VIEWS

```
x = rand(1_000)

@views function foo(x)
    x1 = x[x .> 0.7]
    x2 = x[x .< 0.5]
    x3 = x[1:500]
    x4 = x[501:1_000]

    x1, x2, x3, x4
end
```

A second application of macros is **to modify how operations are computed**, which is the focus of the current section. This lets developers bundle sophisticated optimization techniques, making advanced solutions widely accessible. As a result, users unfamiliar with a method's underlying complexity can focus on choosing the most suitable computational approach, rather than grappling with the implementation details.

While macros are powerful tools, they're not without their limitations. Their black-box nature means that **misuse of macros can lead to unexpected results or compromise computational safety**. That's why it's crucial to identify the right scenarios for each macro. Although this requires some upfront work, it's considerably less demanding than implementing the functionality from scratch.

MACROS APPLIED IN FOR-LOOPS

One distinctive feature of Julia is its ability to execute for-loops with exceptional speed. In fact, carefully optimized for-loops can reach peak performance within the language. This efficiency stems from the versatility of for-loops, which lets users fine-tune them for their specific needs. As a result, it's no surprise that one prominent application of macros is to customize how for-loops are computed.

To illustrate this usage, let's consider the `@inbounds` macro. To understand what this macro accomplishes, we must first understand how for-loops behave in Julia. By default, the language implements **bounds checking**: when an element `x[i]` is accessed during the *i*-th iteration, Julia

verifies that *i* falls within the valid range of indices for `x`. This built-in mechanism safeguards against errors and security issues caused by out-of-bounds access.

While bounds checking prevents bugs, it comes at a performance cost: the additional checks not only introduce computational overhead, but also limit the compiler's ability to implement certain optimizations. In situations where iterations are guaranteed to stay within an array's bounds, these safety checks become redundant. Consequently, we can safely boost performance by disabling bounds checking with the `@inbounds` macro.

> **Trade-Offs Entailed by @inbounds**
>
> The `@inbounds` macro perfectly illustrates both the power and risks associated with macro usage. When applied judiciously, it can yield substantial performance gains, especially when multiple slices are involved.
>
> Despite this, disabling bounds checking simultaneously renders code unsafe: it increases the risk of crashes and silent errors, additionally creating security vulnerabilities. In this context, `@inbounds` shifts the responsibility of applying the macro onto the user, who must be absolutely certain that the iteration range lies within the arrays' bounds.

@INBOUNDS AS AN EXAMPLE

A macro that modifies the entire structure of the for-loop requires its inclusion at the beginning of the for-loop. For instance, to disable bounds checking for every index accessed within a for-loop, we need to prepend the for-loop with `@inbounds`.

@INBOUNDS PREPENDED TO FOR-LOOP

```julia
x = rand(1_000)

function foo(x)
    output = 0.0

    @inbounds for i in eachindex(x)
                  a       = log(x[i])
                  b       = exp(x[i])
                  output += a / b
    end

    return output
end
```

```
julia> @btime foo($x)
  5.825 μs (0 allocations: 0 bytes)
```

> **Alternative Application of @inbounds**

We can alternatively apply `@inbounds` individually to any specific line within the for-loop. This possibility is specific to macros like `@inbounds`, which aren't exclusive to for-loops. Rather, it can be applied to any expression that access elements of a collection.

@INBOUNDS INDIVIDUALLY ADDED

```
x = rand(1_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        @inbounds a       = log(x[i])
        @inbounds b       = exp(x[i])
                  output += a / b
    end

    return output
end
```

```
julia> @btime foo($x)
  5.940 µs (0 allocations: 0 bytes)
```

The performance advantages of `@inbounds` come not only from eliminating bounds checking itself, but also from giving the compiler more freedom to implement additional optimizations. This is because bounds checking is essentially a conditional statement, where the iteration is executed only if all indices are within range. As we'll see in the next sections, conditional statements limit the compiler's ability to apply the so-called SIMD instructions, which are a form of parallelism within a single core.

MACROS COULD BE APPLIED AUTOMATICALLY OR BE DISREGARDED BY THE COMPILER

The influence of a macro on code execution isn't always predictable. Because the compiler has the final say on the optimization strategy, the macro might actually have no impact at all. This would occur if the compiler is already applying the optimization suggested, or if it determines that the macro's recommendation is unhelpful and simply ignores it. In practice, you'll notice that a macro is inconsequential when execution time remains essentially unchanged after its introduction. [1] Scenarios like this can arise with `@inbounds`, as we show below.

REDUNDANT MACROS

The compiler could determine on its own that a for-loop is safe and therefore disable bounds check. In those cases, `@inbounds` becomes redundant. This behavior typically occurs in simple cases, such as when iterating over a single collection `x` using `eachindex(x)`.

NO MACRO

```
x = rand(1_000)

function foo(x)
    output = 0.0

    for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.206 μs (0 allocations: 0 bytes)
```

ADDING @INBOUNDS

```
x = rand(1_000)

function foo(x)
    output = 0.0

    @inbounds for i in eachindex(x)
        output += log(x[i])
    end

    return output
end
```

```
julia> @btime foo($x)
  3.266 μs (0 allocations: 0 bytes)
```

DISREGARDED MACRO

A macro can also serve as a lightweight hint that the compiler is free to disregard. A user might introduce a macro to indicate that the conditions for a particular optimization are met, giving the compiler permission to consider more aggressive strategies. The compiler then analyzes the surrounding code, evaluates whether the suggested approach is actually advantageous, and implements it only when it expects a performance gain. If its analysis suggests that the optimization would be counterproductive, the compiler simply discards the hint. In this role, macros guide the compiler toward potential optimizations, without imposing mandatory directives.

A prime example of a macro that plays this advisory role is `@simd`. This macro suggests the application of **SIMD instructions**, an optimizaiton that will be explored in upcoming sections. When `@simd` is added to a for-loop, nonetheless, the compiler retains full autonomy in deciding whether to implement it.

In the example below, the compiler concludes that SIMD would likely degrade performance, thus ignoring its application. The decision is reflected by the execution time, which remain the same with and without the `@simd` annotation. [2]

WITHOUT @SIMD

```
x = rand(2_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = if (200_000 > i > 100_000)
                        x[i] * 1.1
                    else
                        x[i] * 1.2
                    end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.128 ms (3 allocations: 15.259 MiB)
```

WITH @SIMD

```
x = rand(2_000_000)

function foo(x)
    output = similar(x)

    @simd for i in eachindex(x)
        output[i] = if (200_000 > i > 100_000)
                        x[i] * 1.1
                    else
                        x[i] * 1.2
                    end
    end

    return output
end
```

```
julia> @btime foo($x)
  1.415 ms (3 allocations: 15.259 MiB)
```

FOOTNOTES

[1.] One can verify that macros are indeed ignored by examining the generated machine code. Because inspecting machine code is beyond the scope of this book, we'll instead use the runtime behavior as an indicator.

[2.] It's possible to confirm that the generated machine code is identical by inspecting the compiler's implemented code.